

AI Assisted Coding

Name : M.Pravalika

H.no : 2303A52347

Batch : 34

Course Code : 23CS002PC304

Assignment Number : 17.2

Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts

Task Description – 1(Student Academic Data Cleaning)

- Task: Use AI assistance to generate a Python script that cleans and prepares a student academic dataset for analysis.

Instructions:

- Handle missing values in marks, attendance, and department.
- Remove duplicate student records based on student ID.
- Standardize text fields such as department names.
- Encode categorical variables like gender and department.

Sample Code:

```
import pandas as pd  
data = pd.read_csv("students.csv")  
print(data.head())
```

Expected Output:

- A cleaned and encoded Pandas DataFrame suitable for academic performance analysis.


```
import pandas as pd
```

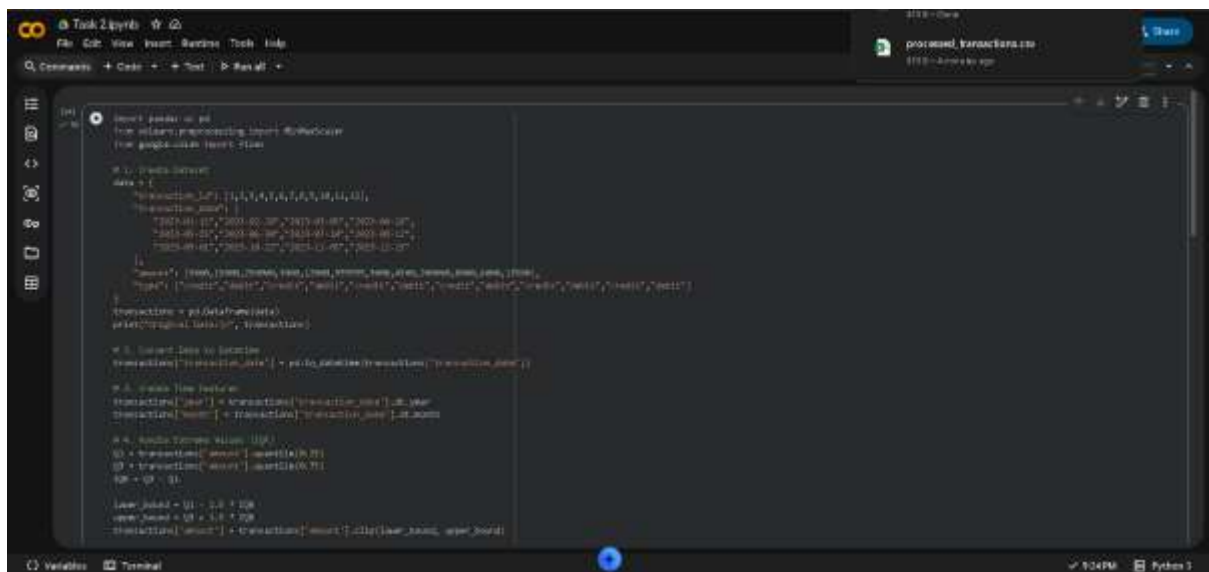
```
from sklearn.preprocessing import MinMaxScaler
```

```
transactions = pd.read_csv("transactions.csv")
```

```
print(transactions.info())
```

Expected Output:

- A transformed dataset with normalized values and newly generated time based features.



```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from datetime import datetime

# 1. Create Dataset
data = [
    "transaction_id", [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "transaction_amount", [
        "2023-01-01", "2023-02-15", "2023-03-05", "2023-04-10",
        "2023-05-20", "2023-06-30", "2023-07-15", "2023-08-10",
        "2023-09-01", "2023-10-20", "2023-11-05", "2023-12-01"
    ],
    "amount", [5000, 10000, 20000, 30000, 40000, 50000, 60000, 70000, 80000, 90000, 100000],
    "type", ["credit", "debit", "credit", "debit", "credit", "debit", "credit", "debit", "credit", "debit", "credit"]
]

transactions = pd.DataFrame(data)
print(transactions.info())

# 2. Convert Date to datetime
transactions["transaction_date"] = pd.to_datetime(transactions["transaction_date"])

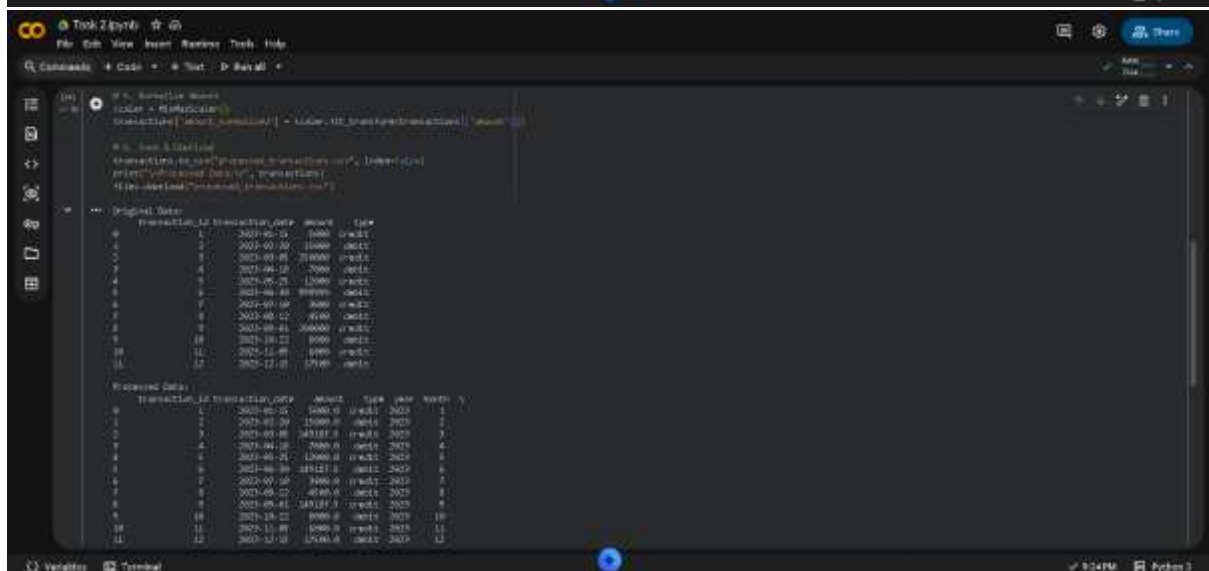
# 3. Create Time Features
transactions["year"] = transactions["transaction_date"].dt.year
transactions["month"] = transactions["transaction_date"].dt.month

# 4. Add Transaction ID as Index
ID = transactions.index
ID = transactions.index.astype(int)
ID = ID + 1

transactions.index = ID

# 5. Add Transaction ID as Index
ID = transactions.index
ID = transactions.index.astype(int)
ID = ID + 1

transactions.index = ID
```

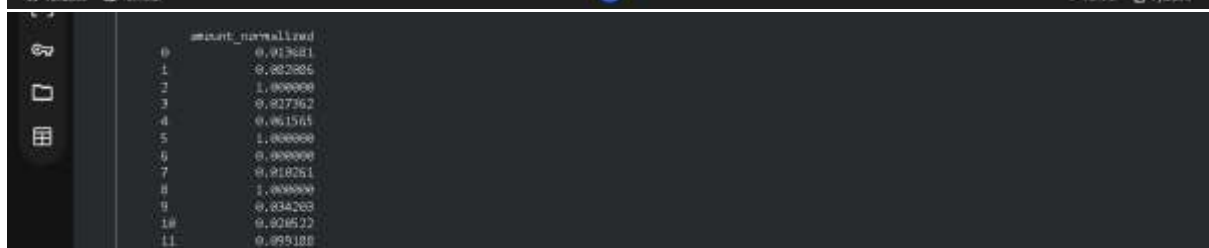


```
# 6. Initialize MinMaxScaler
scaler = MinMaxScaler()

transactions["amount_normalized"] = scaler.fit_transform(transactions["amount"])

# 7. View & Display
transactions.to_csv("processed_transactions.csv", index=False)
print("Processed Data:", transactions)
file_location = "processed_transactions.csv"
```

transaction_id	transaction_date	amount	type
1	2023-01-01	5000	credit
2	2023-02-15	10000	debit
3	2023-03-05	20000	credit
4	2023-04-10	30000	debit
5	2023-05-20	40000	credit
6	2023-06-30	50000	debit
7	2023-07-15	60000	credit
8	2023-08-10	70000	debit
9	2023-09-01	80000	credit
10	2023-10-20	90000	debit
11	2023-11-05	100000	credit
12	2023-12-01	110000	debit



```
amount_normalized
```

amount_normalized	
0	0.013681
1	0.027362
2	0.054724
3	0.109448
4	0.164172
5	0.218896
6	0.273620
7	0.328344
8	0.383068
9	0.437792
10	0.492516
11	0.547240

Task 3 – (Environmental Sensor Data Preparation)

- Task: Clean and preprocess environmental sensor data using AI-assisted scripts.

Instructions:

- Handle missing values in temperature, humidity, and air_quality_index.
- Scale numerical features using standard scaling.
- Encode categorical fields such as sensor_status.
- Split the dataset into training and testing subsets.

Sample Code:

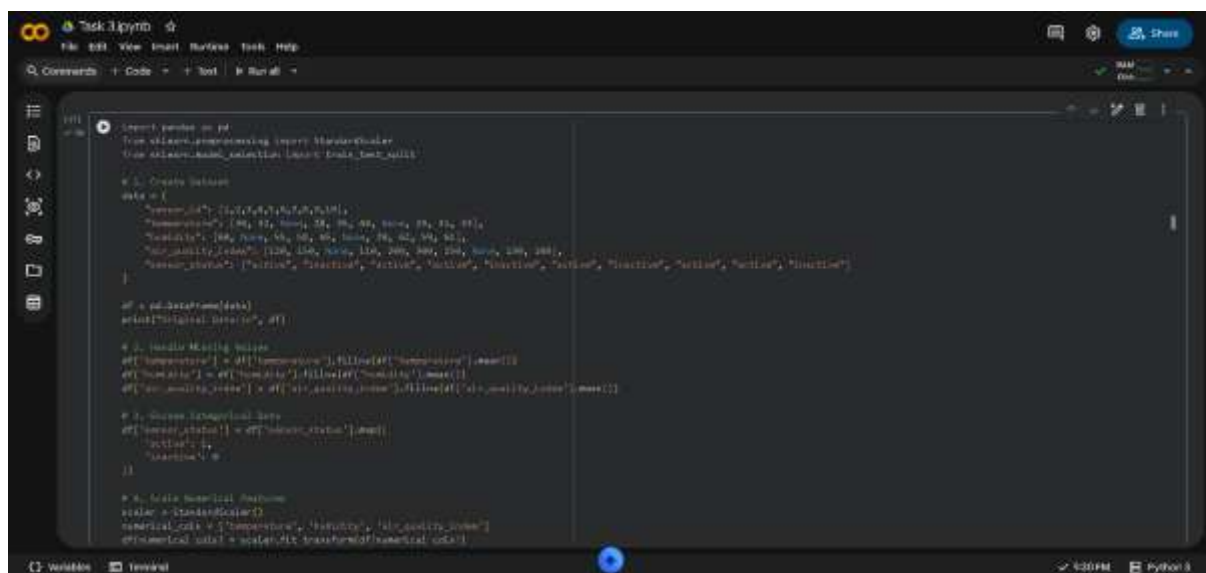
```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
data = pd.read_csv("sensor_data.csv")
```

Expected Output:

- A fully preprocessed dataset ready for predictive modeling tasks.



```
Task 3.ipynb
File Edit View Insert Cell Help
In [ ]:
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# 1. Create Dataset
data = {
    "sensor_id": [101, 102, 103, 104, 105],
    "temperature": [25, 28, None, 22, 26, 29, 27, 24],
    "humidity": [65, 70, 68, 62, 72, 69, 71, 66],
    "air_quality_index": [120, 130, None, 110, 140, 125, 135, 115],
    "sensor_status": ["Active", "Inactive", "Active", "Active", "Inactive", "Active", "Inactive", "Active"]
}

df = pd.DataFrame(data)
print("Initial Dataset:", df)

# 2. Handle Missing Values
df["temperature"] = df["temperature"].fillna(df["temperature"].mean())
df["humidity"] = df["humidity"].fillna(df["humidity"].max())
df["air_quality_index"] = df["air_quality_index"].fillna(df["air_quality_index"].mean())

# 3. Encode Categorical Data
df["sensor_status"] = df["sensor_status"].map(
    {"Active": 1,
     "Inactive": 0}
)

# 4. Scale Numerical Features
scaler = StandardScaler()
numerical_cols = ["temperature", "humidity", "air_quality_index"]
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

Out[ ]:
[]
```

```

# 3. Split Dataset
X = df.drop(columns='sensor_id') # Features
y = df['sensor_status'] # Target

# 4. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 5. Feature Scaling
print(f"Normalized Data (0 to 1):")
print(f"Training Set Shape: {X_train.shape}")
print(f"Testing Set Shape: {X_test.shape}")

# Original Data:
sensor_id temperature humidity air_quality_index sensor_status
0 1 85.0 65.0 120.0 inactive
1 2 72.0 75.0 150.0 inactive
2 3 90.0 55.0 80.0 active
3 4 10.0 10.0 110.0 active
4 5 95.0 85.0 200.0 inactive
5 6 40.0 5.0 900.0 inactive
6 7 90.0 70.0 250.0 inactive
7 8 10.0 60.0 90.0 active
8 9 81.0 66.0 100.0 active
9 10 77.0 81.0 100.0 inactive

# Normalized Data:
sensor_id temperature humidity air_quality_index sensor_status
0 1 -0.000000 -0.104222 -0.557123 1
1 2 -0.000000 0.000000 -0.000000 0
2 3 0.000000 -1.000000 0.000000 1
3 4 -1.000000 -0.000000 -1.000000 1
4 5 0.000000 0.000000 0.000000 0
5 6 0.000000 0.000000 0.000000 0
6 7 0.000000 0.000000 0.000000 0
7 8 -1.000000 0.000000 0.000000 1
8 9 0.000000 -0.000000 -0.000000 1
9 10 0.000000 0.000000 0.000000 0

Training Set Shape: (8, 4)
Testing Set Shape: (2, 4)

```

Task 4 – (Real-Time Application: Smart City Traffic Data Cleaning)

• Scenario:

A smart city system collects traffic data from multiple sensors, resulting in noisy and inconsistent records.

Requirements:

- Standardize road and location names.
- Fill missing traffic density values using appropriate statistical methods.
- Remove duplicate sensor readings.
- Normalize speed and vehicle count metrics.
- Generate a brief summary comparing dataset quality before and after cleaning

Sample Code:

```

import pandas as pd

traffic = pd.read_csv("traffic_data.csv")

print(traffic.describe())

```

Expected Output:

- A cleaned and normalized traffic dataset with an accompanying

data-quality improvement summary.

```
Task 4.ipynb
File Edit View Insert Run/Tools Task Help
⌕ Commands + Code + Test ▶ Run All
Python 3

import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# 1. Create Initial Dataset
data = {
    "sensor_id": [1,2,3,4,5,6,7,8,9,10],
    "road_name": ["M1 Road", "M20 Road", "M10 Road", "Ring Road", "M1 Road", "M20 Road", "M10 Road", "Ring Road", "M1 Road", "M20 Road"],
    "location": ["London", "London", "London", "London", "London", "London", "London", "London", "London", "London"],
    "traffic_density": [100, 80, 60, 50, 40, 30, 20, 10, 90, 70],
    "speed": [100, 80, 60, 50, 40, 30, 20, 10, 90, 70],
    "vehicle_count": [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]
}

traffic = pd.DataFrame(data)

print("Original Dataset:", traffic)

# 2. Data Quality Check (Cleaning)
print("\n• Before cleaning dataset")
print("Missing Values:", traffic.isnull().sum())
print("Duplicate Rows:", traffic.duplicated().sum())

# 3. Drop missing last column
traffic["vehicle_count"] = traffic["vehicle_count"].fillna(0)
traffic["location"] = traffic["location"].str.strip()

# Remove duplicates
traffic["sensor_id"] = traffic["sensor_id"].drop_duplicates()

# 4. Handle Missing Values
traffic["traffic_density"] = traffic["traffic_density"].fillna(
    traffic["traffic_density"].mean()
)

# 5. Remove Duplicates
traffic = traffic.drop_duplicates()

# 6. Normalize Numerical Features
scaler = MinMaxScaler()
traffic[["sensor_id", "traffic_density"]] = scaler.fit_transform(
    traffic[["sensor_id", "traffic_density"]]
)

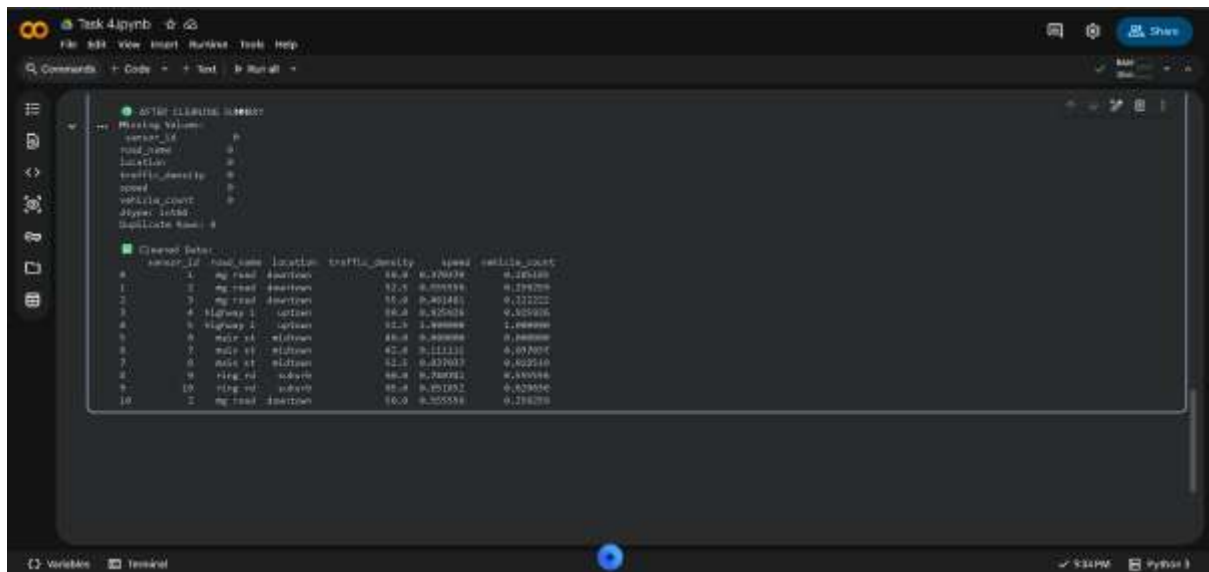
# 7. Data Quality After Cleaning
print("\n• After cleaning dataset")
print("Missing Values:", traffic.isnull().sum())
print("Duplicate Rows:", traffic.duplicated().sum())

# 8. Final Dataset
print("\n• Cleaned Dataset:", traffic)

# Save cleaned dataset
traffic.to_csv("cleaned_traffic_data.csv", index=False)

# Original Data:
# sensor_id road_name location traffic_density speed vehicle_count
# 0 1 M1 Road London 100.0 100 1000
# 1 2 M20 Road London 80.0 80 200
# 2 3 M10 Road London 60.0 60 300
# 3 4 Ring Road London 50.0 50 400
# 4 5 M20 Road London 40.0 40 500
# 5 6 M1 Road London 30.0 30 600
# 6 7 Ring Road London 20.0 20 700
# 7 8 M20 Road London 10.0 10 800
# 8 9 M1 Road London 90.0 90 900
# 9 10 M20 Road London 70.0 70 1000

# Before cleaning dataset
Missing Values:
sensor_id 0
road_name 0
location 0
traffic_density 0
speed 0
vehicle_count 0
Dropped Rows: 0
```



Task 5 – (Social Media Text Data Cleaning and Feature Extraction)

• Task:

Use AI-assisted Python scripting to clean and preprocess social media text data for sentiment analysis.

Instructions:

- Remove duplicate posts and null text entries.
- Clean text by removing URLs, special characters, and extra spaces.
- Convert text to lowercase and perform basic tokenization.
- Remove stopwords and perform stemming or lemmatization. Generate basic text features such as word count and sentiment score.

Sample Code:

```
import pandas as pd
import re

posts = pd.read_csv("social_media_posts.csv")
print(posts.head())
```

Expected Output:

A cleaned text dataset with extracted features suitable for sentiment analysis models.


```
Task E.gymn
File Edit View Insert Run/Debug Tools Help
Commands Code Run Run All

Original Data:
sent_id text
0 1 I love this product! https://example.com
1 2 Worst service ever!!!
2 3 ...
3 4 Amazing experience, will buy again!!!
4 5 Check this out! https://example.com
5 6 Not good, very bad service
6 7 OK product, nothing special
7 8 I love this product! https://example.com
8 9 Worst service ever!!!

Cleaned & Preprocessed Data:
sent_id text
0 1 I love this product! https://example.com
1 2 Worst service ever!!!
2 3 ...
3 4 Amazing experience, will buy again!!!
4 5 Check this out! https://example.com
5 6 Not good, very bad service
6 7 OK product, nothing special
7 8 I love this product! https://example.com

class Tweet:
    def __init__(self, tweet_id, text):
        self.tweet_id = tweet_id
        self.text = text

    def __str__(self):
        return f'Tweet {self.tweet_id}: {self.text}'

# Create a list of Tweet objects
tweets = [
    Tweet(0, '1 I love this product! https://example.com'),
    Tweet(1, '2 Worst service ever!!!'),
    Tweet(2, '3 ...'),
    Tweet(3, '4 Amazing experience, will buy again!!!'),
    Tweet(4, '5 Check this out! https://example.com'),
    Tweet(5, '6 Not good, very bad service'),
    Tweet(6, '7 OK product, nothing special'),
    Tweet(7, '8 I love this product! https://example.com')
]

# Preprocess the tweets
preprocessed_tweets = []
for tweet in tweets:
    preprocessed_tweets.append(tweet.text)

# Calculate sentiment scores
sentiment_scores = {}
for tweet_id, tweet_text in enumerate(preprocessed_tweets):
    sentiment_scores[tweet_id] = calculate_sentiment_score(tweet_text)

# Print the sentiment scores
print('tweet_id\tword_count\t sentiment_score')
0 I love product 2 0.000000e+00
1 worst service 2 -1.000000e+00
3 most expert top 3 0.000000e+00
4 check service 3 0.000000e+00
5 good bad service 3 3.951129e-07
6 ok product special 3 0.285714e+01
7 I love product 2 0.000000e+00
```